

Módulo 2. DataOps con DVC y validación de datos

Curso MLOps/DataOps · ANBAN

José Picón

2026

1. Qué es DataOps y por qué te importa

Antes de entrar en herramientas, asegurémonos de entender qué es DataOps. La definición corta:

DataOps es la disciplina que aplica las prácticas de DevOps al ciclo de vida del dato.

Eso significa: automatización, versionado, observabilidad y calidad desde el momento en que el dato entra al sistema hasta que llega al modelo. En la práctica, cuatro principios:

- 1. Orquestación.** Defines tu pipeline de datos como un grafo declarativo (ingesta → limpieza → transformación → validación → publicación). Un solo comando lo reproduce entero.
- 2. Observabilidad.** En cada momento sabes en qué paso del pipeline estás, cuánto ha tardado, qué datos ha producido y si algún paso ha fallado.
- 3. Automatización.** Si tienes que ejecutar un paso a mano cada vez, antes o después se te olvidará. Todo se automatiza.
- 4. Versionado.** Datos, código y parámetros. Los tres tienen que poder volverse atrás juntos a un estado anterior cualquiera.

En este módulo vamos a centrarnos en los puntos 1 (orquestación) y 4 (versionado) usando **DVC**, y en el punto de calidad de datos con una implementación inspirada en **Great Expectations**.

2. La herramienta principal: DVC

2.1 Qué es DVC

DVC son las siglas de Data Version Control. Es una herramienta de código abierto que extiende Git para que pueda manejar ficheros muy grandes sin almacenarlos directamente en el repositorio. Su modelo mental es sencillo:

- En **Git** se guarda solo un puntero pequeño (un fichero `.dvc` de unos pocos bytes) que contiene el hash criptográfico del dato real.
- El **dato real** (que puede pesar gigabytes) se guarda en un almacenamiento externo: un bucket S3, un MinIO local, Google Cloud Storage, Azure Blob, un disco compartido, etc.

Cuando un compañero clona el repositorio, Git le baja solo los punteros. Para recuperar los datos reales ejecuta `dvc pull` y DVC los descarga del almacenamiento externo usando los hashes como referencia.

2.2 Por qué no basta con Git para datos

Git se diseñó para texto plano. Tres problemas si guardas datos en Git:

1. **Crece sin control.** Git almacena cada versión del binario entera, no diferencias. Un CSV de 500 MB que cambias diez veces ocupa 5 GB en el repositorio.
2. **Los clones se vuelven imposibles.** Cualquier `git clone` baja todo el histórico.
3. **GitHub bloquea ficheros grandes.** El límite duro está en 100 MB por fichero, y a partir de 50 MB ya recibes advertencias.

DVC resuelve estos tres problemas a la vez: Git solo trackea metadatos, los datos viven aparte.

2.3 Alternativas a DVC

No es la única herramienta. Te interesa conocer las opciones:

Herramienta	Idea	Cuándo elegirla
DVC	Puntero en git + blob en remoto	Equipos pequeños/medianos, integración con Git, on-premises
LakeFS	Git para data lakes enteros	Si ya trabajas con S3/GCS a escala
Delta Lake	Tablas versionadas sobre Spark/Databricks	Stack Databricks
Apache Iceberg	Formato de tabla con snapshots	Data lakes modernos, OLAP

En este curso usamos DVC porque encaja con Git, funciona en local con MinIO sin necesidad de cloud, y enseña los conceptos generales muy bien.

2.4 Instalación de DVC

DVC se instala con `pip` (el gestor de paquetes de Python). Como en el curso usamos Docker, **no tienes que instalarlo en tu sistema**: el script `run.sh` lo instala dentro de un entorno aislado cuando ejecutas el lab. Pero si quieres tenerlo a mano fuera del lab:

macOS, Linux, Windows con WSL

Abre tu terminal y ejecuta:

```
pip install "dvc[s3]==3.55.2"
```

La sintaxis `dvc[s3]` significa “instala DVC con el extra `s3`”. Ese extra es el plugin que permite hablar con MinIO y otros almacenes compatibles con S3. Sin él, solo podrías usar DVC con discos locales.

Verifica:

```
dvc --version
```

Debe imprimir algo como `3.55.2`.

Windows nativo

Si no usas WSL, primero asegúrate de tener Python instalado. Abre PowerShell:

```
python --version
```

Si no responde, instala Python desde Microsoft Store buscando “Python 3.11” o desde python.org.
Después:

```
pip install "dvc[s3]==3.55.2"  
dvc --version
```

3. La herramienta secundaria: Great Expectations (y su alternativa Pandera)

3.1 Qué es Great Expectations

Great Expectations (GE) es una librería de Python para escribir “expectations”: reglas que tus datos deben cumplir. Por ejemplo:

- La columna `edad` debe tener valores entre 0 y 120.
- La columna `email` no puede tener valores nulos.
- La columna `pais` solo puede tomar valores de un catálogo cerrado.

Cuando un dataset llega a tu pipeline, lo pasas por GE y si alguna expectation falla, el pipeline se detiene. Esto evita que datos malos se propaguen aguas abajo y rompan modelos en producción.

3.2 Por qué no usamos Great Expectations directamente

GE es muy potente pero su API cambia mucho entre versiones. En un curso de dos días, perderíamos demasiado tiempo peleándonos con la configuración. Por eso vamos a implementar las mismas ideas con **pandas puro**: el código es más simple, sin librerías que cambien, y los conceptos son idénticos.

Si quieres usar GE de verdad después del curso, los conceptos se traducen uno a uno.

3.3 La alternativa ligera: Pandera

Otra opción muy popular es **Pandera**, una librería más ligera que GE pero con la misma idea. Se integra muy bien con pandas y polars. Definición de schema con Pandera:

```
import pandera as pa

schema = pa.DataFrameSchema({
    "age": pa.Column(int, pa.Check.in_range(0, 120)),
    "email": pa.Column(str, nullable=False),
})
```

No la usamos en el lab para no añadir dependencias, pero es buena opción para tus proyectos.

4. Las cinco dimensiones de calidad del dato

Antes de empezar a trackear datos, conviene tener un marco mental para pensar en su calidad. Cinco dimensiones que cualquier proyecto serio debería medir:

Dimensión	Pregunta que responde
Compleitud	¿Cuántos valores nulos hay donde no debería haberlos?
Validez	¿Los valores están dentro del rango o dominio esperado?
Unicidad	¿Existen duplicados?
Consistencia	¿Cuadran los datos entre fuentes?
Freshness (frescura)	¿El dato es de cuándo dice ser?

En el lab vamos a comprobar **compleitud** (no nulos en `income`), **validez** (rango de `age`, dominio de `workclass`) y **schema** (columnas presentes).

5. Antes de la práctica: requisitos

Para hacer el lab necesitas:

1. **El laboratorio del curso arrancado** (`./setup.sh` ya ejecutado). Si no, vuelve al módulo 0.
2. **Una terminal abierta** en la carpeta `Anbam-curso/`.
3. **La carpeta `labs/lab1_dataops/`** con los ficheros del lab.

Comprueba que tienes lo que hace falta:

```
cd labs/lab1_dataops
ls
```

Deberías ver:

```
params.yaml  README.md  run.ps1  run.sh  src
```

Si todo está, sigue al ejercicio.

6. Ejercicio: montar un pipeline reproducible con DVC

Objetivo del ejercicio: practicar el ciclo entero de DVC partiendo de un repositorio vacío. Al terminar tendrás un pipeline declarativo que valida y preprocesa el dataset UCI Adult, con el dato versionado por DVC y los artefactos generados en parquet.

6.1 Cómo hacer este ejercicio

Tienes dos formas:

- **Guiada con `run.sh`**. El script te lleva paso a paso, ejecutando cada comando y explicándolo. Tú lees y pulsas Enter. Recomendado para la primera vez.
- **A mano**. Sigues los pasos de este capítulo tecleando cada comando tú. Recomendado para la segunda vez, cuando quieras entender qué hace cada línea.

Las dos formas hacen exactamente lo mismo.

6.2 Paso 1 — Inicializar Git

DVC vive dentro de Git: extiende Git, no lo reemplaza. Por eso lo primero es crear un repositorio Git en la carpeta del lab.

```
git init -q
```

Verifica:

```
ls -la .git
```

Tiene que existir una carpeta `.git`. Esa es la carpeta interna de Git con todo el histórico.

A continuación configura tu identidad (Git la pide para los commits):

```
git config --local user.email "alumno@anban.es"  
git config --local user.name "Alumno ANBAN"
```

El flag `--local` significa que esta configuración solo aplica en este repositorio, no en otros que tengas.

Haz un primer commit con lo que ya hay:

```
git add -A  
git commit -m "snapshot inicial"
```

`git add -A` añade todos los ficheros al “staging area”. `git commit` crea un punto en el histórico con esos cambios.

6.3 Paso 2 — Inicializar DVC

```
dvc init -q
```

El flag `-q` significa “modo silencioso”. DVC crea una carpeta `.dvc/` y un fichero `.dvcignore`. Mira lo que aparece:

```
ls -la
```

Verás `.dvc/` y `.dvcignore`. Estos son los ficheros que DVC necesita para funcionar. Los commiteamos al repositorio:

```
git add .dvc .dvcignore
git commit -m "inicializa dvc"
```

6.4 Paso 3 — Configurar el remoto MinIO

DVC necesita saber dónde guardar los blobs grandes. Vamos a decirle que use el bucket `datasets` de nuestro MinIO local.

```
dvc remote add -d minio s3://datasets/lab1
```

Descomponemos este comando:

- `dvc remote add` : registra un nuevo remoto.
- `-d` : marca este remoto como el “default” (el que se usa si no se especifica otro).
- `minio` : es el nombre que le damos al remoto (puedes ponerle el que quieras).
- `s3://datasets/lab1` : la URL del remoto. El protocolo `s3://` funciona con MinIO porque MinIO habla S3 nativo.

A continuación le decimos a DVC dónde está el servidor MinIO:

```
dvc remote modify minio endpointurl http://localhost:9000
```

Esto modifica la propiedad `endpointurl` del remoto `minio`. Lo guarda en `.dvc/config`, que **sí se commitea en Git** (no contiene secretos, solo la URL).

Y ahora las credenciales:

```
dvc remote modify --local minio access_key_id minio
dvc remote modify --local minio secret_access_key minio12345
```

Fíjate en el flag `--local`. Eso es **muy importante**: con `--local` DVC guarda esta configuración en `.dvc/config.local`, que está en `.gitignore` y **no se commitea**. Las credenciales nunca, jamás, bajo ningún concepto deben terminar en Git.

Verifica:

```
cat .dvc/config
cat .dvc/config.local
```

El primero tiene la URL pública. El segundo tiene las credenciales. El primero está en Git, el segundo está ignorado.

Commitea el config público:

```
git add .dvc/config
git commit -m "remoto minio configurado"
```

6.5 Paso 4 — Trackear el dataset

Vamos a copiar el dataset del curso a una carpeta `data/raw/` y decirle a DVC que lo trackee.

```
mkdir -p data/raw
cp ../../datasets/adult/adult.csv data/raw/adult.csv
```

`mkdir -p` crea la carpeta y todas las intermedias si hace falta. El `cp` copia el dataset desde la carpeta común `datasets/` del curso.

Verifica:

```
ls -lh data/raw/
```

Debe aparecer `adult.csv` con un tamaño de unos 3,8 MB.

Ahora viene el comando clave de DVC:

```
dvc add data/raw/adult.csv
```

¿Qué hace este comando? Tres cosas:

1. Calcula el hash MD5 del fichero.
2. Mueve el fichero a la **caché interna de DVC** (en `.dvc/cache/`), nombrándolo por su hash.

3. Crea en su lugar un **puntero**: el fichero `adult.csv.dvc` que contiene los metadatos del original.

Mira los ficheros que ha creado:

```
ls -la data/raw/  
cat data/raw/adult.csv.dvc  
cat data/raw/.gitignore
```

El `.dvc` contiene el hash y el tamaño. El `.gitignore` excluye el CSV real para que no se cuele en Git. Todo esto lo hace DVC automáticamente.

Commitea el puntero:

```
git add data/raw/adult.csv.dvc data/raw/.gitignore  
git commit -m "trackea dataset"
```

Ahora súbelo al remoto:

```
dvc push
```

Esto sube los blobs desde la caché local a MinIO. Verifica abriendo `http://localhost:9001` en el navegador: entra al bucket `datasets/`, luego en `lab1/`, y verás una carpeta con un fichero nombrado por su hash. Eso es DVC en su mínima expresión.

6.6 Paso 5 — Escribir las expectations

El fichero `src/ge_validate.py` ya está en el repositorio. Vamos a mirar su estructura:

```
cat src/ge_validate.py
```

El fichero define cuatro funciones de validación:

```

def expect_columns(df):
    # Comprueba que están todas las columnas esperadas
    missing = set(EXPECTED_COLUMNS) - set(df.columns)
    assert not missing, f"faltan columnas: {missing}"

def expect_age_range(df, lo=17, hi=90):
    # Comprueba que age está dentro del rango [17, 90]
    bad = df[(df["age"] < lo) | (df["age"] > hi)]
    assert bad.empty, f"{len(bad)} filas con age fuera de [{lo}, {hi}]"

def expect_workclass_in_domain(df):
    # Comprueba que workclass solo tiene valores conocidos
    vals = set(df["workclass"].dropna().unique())
    extras = vals - WORKCLASS_DOMAIN
    assert not extras, f"valores inesperados: {extras}"

def expect_income_not_null(df):
    # Comprueba que income no tiene nulos
    nulls = df["income"].isna().sum()
    assert nulls == 0, f"income tiene {nulls} nulos"

```

Cada función es una **expectation**: si todo está bien, no hace nada; si algo falla, lanza una excepción que detiene el pipeline.

Ejecuta el validador a mano para ver qué pasa con el dataset bueno:

```
python src/ge_validate.py data/raw/adult.csv
```

Salida esperada:

```

[OK] schema (las 15 columnas esperadas están presentes)
[OK] age dentro de [17, 90]
[OK] workclass dentro del dominio conocido
[OK] income sin nulos

TODAS LAS EXPECTATIVAS PASADAS

```

6.7 Paso 6 — Crear el pipeline

DVC tiene su propio formato declarativo para pipelines: el fichero `dvc.yaml`. Vamos a crearlo. Abre tu editor (puede ser `nano`, `vim`, VS Code, lo que prefieras) y crea `dvc.yaml` con este contenido:

```

stages:
  validate:
    cmd: python src/ge_validate.py data/raw/adult.csv
    deps:
      - data/raw/adult.csv
      - src/ge_validate.py
  preprocess:
    cmd: python src/preprocess.py
    deps:
      - data/raw/adult.csv
      - src/preprocess.py
      - params.yaml
    outs:
      - data/processed/train.parquet
      - data/processed/test.parquet

```

Vamos a entender qué dice este YAML:

- **stages:** abre la lista de etapas (stages) del pipeline.
- **validate:** primera etapa, llamada “validate”.
 - **cmd:** comando que se ejecuta cuando se lanza esta etapa.
 - **deps:** dependencias. Si cualquiera de estos ficheros cambia, la etapa se vuelve a ejecutar; si no cambia, se salta.
- **preprocess:** segunda etapa. Depende del CSV crudo, del script de preprocesado y de los parámetros.
 - **outs:** salidas. DVC trackea automáticamente estos ficheros como outputs de esta etapa.

Ejecuta el pipeline entero:

```
dvc repro
```

DVC mira las dependencias, ejecuta primero `validate` y luego `preprocess`, y genera `data/processed/train.parquet` y `data/processed/test.parquet`. Verifica:

```
ls -lh data/processed/
```

6.8 Paso 7 — Ver la caché por dependencias en acción

Vuelve a ejecutar el pipeline:

```
dvc repro
```

Esta vez verás algo así:

```
'data/raw/adult.csv.dvc' didn't change, skipping
Stage 'validate' didn't change, skipping
Stage 'preprocess' didn't change, skipping
Data and pipelines are up to date.
```

DVC ha comparado los hashes de las dependencias con los de la ejecución anterior. Como nada ha cambiado, no ejecuta nada. Esto es la **caché por dependencias**, y en pipelines grandes ahorra horas.

Cambia un parámetro y observa qué pasa. Edita `params.yaml` (con `nano params.yaml` por ejemplo) y modifica `test_size` de `0.2` a `0.3`. Guarda. Ejecuta:

```
dvc repro
```

Ahora dice:

```
Stage 'validate' didn't change, skipping
Running stage 'preprocess'
```

Solo se reejecuta `preprocess`, porque solo cambió una dependencia suya (`params.yaml`). `validate` se salta. En pipelines de decenas de etapas, esta inteligencia ahorra mucho tiempo.

Devuelve `test_size` a `0.2` y ejecuta `dvc repro` para dejar el estado limpio.

6.9 Paso 8 — Romper el dato a propósito

Esta es la parte más importante del ejercicio. Vamos a corromper el CSV y comprobar que el validador nos protege.

Edita la línea 5 del CSV. La forma rápida desde la terminal:

```
python3 -c "
with open('data/raw/adult.csv') as f: lines = f.readlines()
parts = lines[4].split(',')
parts[0] = '200'
lines[4] = ','.join(parts)
with open('data/raw/adult.csv','w') as f: f.writelines(lines)
print('Linea 5 modificada: age=200')
"
```

Ahora ejecuta solo el validador. Queremos ver el fallo:

```
python src/ge_validate.py data/raw/adult.csv
```

Debe fallar con:

```
[OK] schema (las 15 columnas esperadas están presentes)
[FALL0] 1 filas con age fuera de [17, 90]
```

Imagina que el dato malo viniera de un cliente real: si el validador no estuviera ahí, el pipeline lo procesaría, entrenaría un modelo con esa basura, y el modelo iría a producción. El validador es el primer cinturón de seguridad.

6.10 Paso 9 — Recuperar el dato bueno

Para recuperar el dato original, lo copiamos otra vez de la fuente y volvemos a trackearlo:

```
cp ../../datasets/adult/adult.csv data/raw/adult.csv
dvc add data/raw/adult.csv
git add data/raw/adult.csv.dvc
git commit -m "restaura dataset original"
dvc repro
```

`dvc repro` debe volver a pasar limpio.

6.11 Paso 10 — Simular un compañero nuevo

Esto demuestra el poder del flujo. Vamos a clonar el repositorio en otra carpeta como si fueras un compañero recién incorporado al proyecto.

```
cd /tmp
git clone ~/Anbam-curso/labs/lab1_dataops nuevo_companero
cd nuevo_companero
ls data/raw/
```

Verás `adult.csv.dvc` y `.gitignore`, pero **no** el CSV grande. Solo el puntero. Ahora recupera los datos:

```
dvc pull
ls -lh data/raw/
```

Ahora sí está el `adult.csv` real, descargado desde MinIO. Reproduce el pipeline:

```
dvc repro
```

Y tienes los parquet generados, exactamente iguales a los originales. Cualquier máquina con acceso al remoto puede reconstruir el dataset y los artefactos exactos a partir del repositorio.

7. Comandos de DVC explicados uno por uno

Como referencia rápida, aquí tienes todos los comandos de DVC que hemos usado, cada uno con su explicación:

Comando	Qué hace
<code>dvc init</code>	Inicializa DVC en un repositorio Git existente. Crea <code>.dvc/</code> y <code>.dvcignore</code> .
<code>dvc add <fichero></code>	Trackea un fichero con DVC. Lo mueve a la caché, crea el puntero <code>.dvc</code> y añade el original al <code>.gitignore</code> .
<code>dvc remote add -d <nombre> <url></code>	Registra un remoto y lo marca como por defecto.
<code>dvc remote modify <nombre> <opción> <valor></code>	Modifica una opción del remoto. Se guarda en <code>.dvc/config</code> (commiteable).
<code>dvc remote modify --local <nombre> <opción> <valor></code>	Como el anterior pero guarda en <code>.dvc/config.local</code> (NO commiteable). Para credenciales.
<code>dvc push</code>	Sube los blobs desde la caché local al remoto.
<code>dvc pull</code>	Descarga los blobs del remoto a la caché local y los enlaza al workspace.
<code>dvc checkout</code>	Sincroniza los ficheros del workspace con los hashes de los <code>.dvc</code> . Útil para revertir cambios.
<code>dvc status</code>	Muestra qué dependencias han cambiado y qué stages necesitan reejecutarse.
<code>dvc repro</code>	Ejecuta el pipeline. Reutiliza las etapas cuyas dependencias no han cambiado.
<code>dvc dag</code>	Imprime el grafo del pipeline (qué stage depende de qué).

8. Pandas y scikit-learn: las librerías de soporte

Los scripts `ge_validate.py` y `preprocess.py` usan dos librerías que conviene conocer:

8.1 Pandas

Pandas es la librería estándar de Python para manipular datos tabulares (filas y columnas). Su tipo principal es el `DataFrame`, muy parecido a una hoja de Excel pero programable.

Las operaciones de pandas que hemos usado en el lab:

```
# Leer un CSV con cabecera proporcionada (porque el original no la tiene):
df = pd.read_csv(path, header=None, names=EXPECTED_COLUMNS, na_values=["?"])

# Filtrar filas:
bad = df[(df["age"] < 17) | (df["age"] > 90)]

# Obtener valores únicos de una columna:
df["workclass"].dropna().unique()

# Contar nulos:
df["income"].isna().sum()

# Eliminar filas con nulos en ciertas columnas:
df.dropna(subset=["workclass", "occupation"])

# Convertir variables categóricas a one-hot:
df = pd.get_dummies(df, columns=cat_cols, drop_first=True)
```

Si no tienes pandas instalado en tu sistema (en el lab lo instala el `run.sh`):

```
pip install pandas pyarrow
```

`pyarrow` es necesario para escribir/leer ficheros parquet.

8.2 Scikit-learn

Scikit-learn (importado como `sklearn`) es la librería estándar de machine learning clásico en Python. En este módulo solo usamos una función:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)
```

Esta función divide los datos en dos conjuntos: entrenamiento (80 %) y test (20 %). El parámetro `stratify=y` asegura que la proporción de clases sea igual en ambos conjuntos (importante para problemas desequilibrados).

Para instalarla:

```
pip install scikit-learn
```

9. Checklist de cierre

Antes de pasar al módulo siguiente, comprueba que puedes responder “sí” a todo:

- ¿Entiendes la diferencia entre lo que Git guarda y lo que DVC guarda?
- ¿Sabes para qué sirve `.dvc/config.local` y por qué no se commitea?
- ¿Has visto que `dvc repro` reutiliza etapas con caché?
- ¿Has roto el dato a propósito y has visto el fallo de validación?
- ¿Has hecho `dvc push` y has visto el blob en MinIO?
- ¿Tienes `data/processed/train.parquet` y `test.parquet` generados?

Si la respuesta es sí a todas, pasas al módulo 3 (MLflow).

10. Para profundizar

Si quieres ir más allá de lo que hemos visto:

- **DVC documentation:** <https://dvc.org/doc>. La sección “Get Started” cubre lo mismo que hemos hecho. La sección “User Guide” entra en pipelines complejos, métricas y experimentos.
- **Great Expectations:** <https://greatexpectations.io>. Si tu empresa tiene catálogo de datos serio.
- **Pandera:** <https://pandera.readthedocs.io>. La alternativa ligera, muy recomendable.
- **LakeFS:** <https://lakefs.io>. Cuando subas a escala de TB.